BILINGUAL INTERNATIONAL AND SUSTAINABLE



Programación Orientada a Objetos

Profesor: Javier Gómez

DSBIS32M

Diego Dael González Acuña

José Fernando González Quiñonez

Paul Daniel Ortiz Contreras

1. Definitions

Error: A severe, unrecoverable problem that occurs at runtime. Usually caused by the environment or system failure, and programs shouldn't try to catch them.

Exception: An abnormal condition or unexpected event that occurs during program execution. Unlike errors, exceptions can be handled programmatically so the application doesn't crash.

2. Characteristics of Errors and Exceptions

Errors: External to the application, fatal, non-recoverable, and unchecked (the compiler doesn't force you to handle them).

Exceptions: Internal to the application, recoverable, can be checked or unchecked, and disrupt normal program flow if left unhandled.

3. Types of Errors

Compile-time Errors: Syntax mistakes, typos, incorrect data types, or missing semicolons detected by the compiler.

Runtime Errors: Issues that occur while the program is running (e.g., division by zero, null pointer dereference, infinite recursion).

Logical Errors: The code compiles and runs without crashing, but produces incorrect output or behaves unexpectedly because of flawed logic.

4. Exception Hierarchies (OOP Example like Java/C#)

Base Class: Throwable (Java) or Exception (C#).

Checked / Compile-time Exceptions: Inherit directly from Exception (excluding RuntimeException). The compiler forces the programmer to handle them using try-catch or throws.

Unchecked / Runtime Exceptions: Inherit from RuntimeException. These represent programming bugs (like indexing out of bounds or division by zero) and don't need explicit handling at compile time.

5. Types of Exceptions: Implicit vs. Explicit

Implicit Exceptions: Thrown automatically by the runtime environment when an illegal operation occurs (e.g., accessing a null reference or dividing by zero).

Explicit Exceptions: Manually thrown by the programmer using keywords like throw when a specific business rule or condition is violated.

6. Exception Handling Mechanisms

Handling Blocks: try, catch, and finally blocks used to isolate risky code, catch specific exception types, and execute cleanup code regardless of whether an exception occurred.

Propagation (Throwing up the call stack): If a method doesn't handle an exception, it automatically propagates up to the calling method until it reaches a matching catch block or crashes the program.

Catching: Intercepting the propagated exception within a suitable scope to log the error, inform the user, or recover gracefully.

Comments

The analysis of error and exception control mechanisms demonstrates that a robust object-oriented design relies not only on correct business logic, but also on the system's ability to anticipate unforeseen failures. The proper separation between fatal system errors and recoverable exceptions allows failures to be isolated and prevents abrupt application crashes, improving user experience and code maintainability.